



Python Programming Fundamentals

File Input and Output

Programming Fundamentals Team

SETU

2026-06-12



Outline

1. Why Files?
2. Opening Files
3. The with Statement
4. Reading Methods
5. Writing Files
6. Exception Handling with Files
7. Practical Example — Log File



Outline

- 1. Why Files?**
2. Opening Files
3. The with Statement
4. Reading Methods
5. Writing Files
6. Exception Handling with Files
7. Practical Example — Log File



1. Why Files?

So far our programs lose all data when they exit. **Files** let us:

- **Persist** data between program runs
- **Share** data between programs
- Process **large datasets** that don't fit in the program code
- Read **configuration** from external files



1. Why Files?

So far our programs lose all data when they exit. **Files** let us:

- **Persist** data between program runs
- **Share** data between programs
- Process **large datasets** that don't fit in the program code
- Read **configuration** from external files

Python can read and write:

- Plain text files (`.txt`)
- CSV files (`.csv`)
- JSON files (`.json`)
- And many more with the right library



Outline

1. Why Files?
- 2. Opening Files**
3. The with Statement
4. Reading Methods
5. Writing Files
6. Exception Handling with Files
7. Practical Example — Log File



2. Opening Files

```
file = open("filename.txt", mode)
```

Mode	Description
"r"	Read (default). Error if file doesn't exist.
"w"	Write. Creates file or overwrites existing content.
"a"	Append. Creates file or adds to end of existing content.
"x"	Exclusive create. Error if file already exists.
"r+"	Read and write.



2. Opening Files

```
file = open("filename.txt", mode)
```

Mode	Description
"r"	Read (default). Error if file doesn't exist.
"w"	Write. Creates file or overwrites existing content.
"a"	Append. Creates file or adds to end of existing content.
"x"	Exclusive create. Error if file already exists.
"r+"	Read and write.

Always close files when done:

```
file = open("data.txt", "r")
content = file.read()
file.close()          # MUST close – frees the resource
```




Outline

1. Why Files?
2. Opening Files
- 3. The with Statement**
4. Reading Methods
5. Writing Files
6. Exception Handling with Files
7. Practical Example — Log File



3. The with Statement

The with statement **automatically closes** the file when the block ends, even if an error occurs.

```
# Preferred way to open files
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
# file is automatically closed here
```



3. The with Statement



3. The with Statement

The with statement **automatically closes** the file when the block ends, even if an error occurs.

```
# Preferred way to open files
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
# file is automatically closed here
```

Writing a file:

```
with open("output.txt", "w") as file:
    file.write("Hello, World!\n")
    file.write("Second line\n")
```

Appending to a file:



3. The with Statement

```
with open("log.txt", "a") as file:  
    file.write("New entry added\n")
```



Outline

1. Why Files?
2. Opening Files
3. The with Statement
- 4. Reading Methods**
5. Writing Files
6. Exception Handling with Files
7. Practical Example — Log File



4. Reading Methods

```
# read() – entire file as one string
with open("poem.txt", "r") as f:
    text = f.read()
    print(text)
```

```
# readline() – one line at a time
with open("data.txt", "r") as f:
    first_line = f.readline()
    second_line = f.readline()
```

```
# readlines() – list of all lines (with \n)
with open("names.txt", "r") as f:
    lines = f.readlines()
    print(lines)    # ['Alice\n', 'Bob\n', 'Carol\n']
```



4. Reading Methods



4. Reading Methods

```
# read() – entire file as one string
with open("poem.txt", "r") as f:
    text = f.read()
    print(text)
```

```
# readline() – one line at a time
with open("data.txt", "r") as f:
    first_line = f.readline()
    second_line = f.readline()
```

```
# readlines() – list of all lines (with \n)
with open("names.txt", "r") as f:
    lines = f.readlines()
    print(lines)    # ['Alice\n', 'Bob\n', 'Carol\n']
```

Most common pattern — iterate line by line:



4. Reading Methods

```
with open("scores.txt", "r") as f:
    for line in f:
        line = line.strip()    # remove \n
        if line:               # skip blank lines
            score = int(line)
            print(f"Score: {score}")
```



Outline

1. Why Files?
2. Opening Files
3. The with Statement
4. Reading Methods
- 5. Writing Files**
6. Exception Handling with Files
7. Practical Example — Log File



5. Writing Files

```
students = [  
    ("Alice", 88),  
    ("Bob", 74),  
    ("Carol", 95),  
    ("Dave", 62),  
]  
  
with open("results.txt", "w") as f:  
    f.write("Student Results\n")  
    f.write("=" * 20 + "\n")  
    for name, score in students:  
        grade = "Pass" if score >= 60 else "Fail"  
        f.write(f"{name:<10} {score:3} {grade}\n")  
  
print("Results saved to results.txt")
```



5. Writing Files



5. Writing Files

```
students = [  
    ("Alice", 88),  
    ("Bob", 74),  
    ("Carol", 95),  
    ("Dave", 62),  
]  
  
with open("results.txt", "w") as f:  
    f.write("Student Results\n")  
    f.write("=" * 20 + "\n")  
    for name, score in students:  
        grade = "Pass" if score >= 60 else "Fail"  
        f.write(f"{name:<10} {score:3} {grade}\n")  
  
print("Results saved to results.txt")
```



5. Writing Files

```
# Reading it back
with open("results.txt", "r") as f:
    print(f.read())
```



Outline

1. Why Files?
2. Opening Files
3. The with Statement
4. Reading Methods
5. Writing Files
- 6. Exception Handling with Files**
7. Practical Example — Log File



6. Exception Handling with Files

Files can fail: the file might not exist, disk might be full, permissions might be denied.

```
try:
    with open("data.txt", "r") as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print("Error: file not found")
except PermissionError:
    print("Error: no permission to read this file")
except IOError as e:
    print(f"I/O Error: {e}")
```



6. Exception Handling with Files



6. Exception Handling with Files

Files can fail: the file might not exist, disk might be full, permissions might be denied.

```
try:
    with open("data.txt", "r") as f:
        content = f.read()
        print(content)
except FileNotFoundError:
    print("Error: file not found")
except PermissionError:
    print("Error: no permission to read this file")
except IOError as e:
    print(f"I/O Error: {e}")
```

Safe file reading function:

```
def read_file(filename):
    """Read a file and return its contents, or None on error."""
```



6. Exception Handling with Files

```
try:
    with open(filename, "r") as f:
        return f.read()
except FileNotFoundError:
    print(f"'{filename}' not found")
    return None

content = read_file("notes.txt")
if content:
    print(content)
```



Outline

1. Why Files?
2. Opening Files
3. The with Statement
4. Reading Methods
5. Writing Files
6. Exception Handling with Files
- 7. Practical Example — Log File**



7. Practical Example — Log File

```
import datetime

def log_event(filename, message):
    """Append a timestamped log entry to a file."""
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(filename, "a") as f:
        f.write(f"[{timestamp}] {message}\n")

def read_log(filename):
    """Print all log entries."""
    try:
        with open(filename, "r") as f:
            for line in f:
                print(line.strip())
    except FileNotFoundError:
        print("No log file yet.")
```



7. Practical Example — Log File

```
log_event("app.log", "Application started")  
log_event("app.log", "User logged in: Alice")  
log_event("app.log", "File processed: data.csv")  
read_log("app.log")
```

Thanks for Watching - Any questions?

